

UNIVERSITY OF BIRMINGHAM

Multi-Objective Portfolio Optimisation using Reinforcement Learning and Evolutionary Algorithms

Matthew Woodcock

Student ID: 2399937

Supervised by Professor Peter Tino

1 Introduction

1.1 The Problem

Portfolio management is the task of deciding how to allocate capital across a set of assets over time. This can be viewed as an optimisation problem, where the goal is to balance returns and risk.

The central challenge in this optimisation problem is that financial markets are non-stationary, with the statistical patterns present in historical data not reliably persisting into the future. This is equivalent to a supervised learning model trained on one data distribution being deployed on a shifted distribution, a problem well understood in machine learning. However, in financial markets this shift can occur suddenly and without warning, making it particularly difficult for any non-ML model to remain well-calibrated over time. For example, the sudden global sell-off in August 2024, triggered by an unexpected rate rise from the Bank of Japan [6], caused sharp, short-lived losses across equity markets. A traditional portfolio management model that does not involve machine learning and has fixed parameters has no mechanism to respond to a sudden change like this.

This motivates the development of a machine learning model capable of dynamically adjusting portfolio allocations in response to changing market conditions.

1.2 Why Classical Methods Fail

The most established classical approach to portfolio optimisation is Markowitz mean-variance optimisation, which selects portfolio weights by maximising expected return for a given level of risk. This approach relies on estimating a fixed covariance matrix and mean return vector from historical data, implicitly assuming that these statistical properties remain stable over time.

This assumption is structurally incompatible with non-stationary markets. When the underlying distribution shifts, the model has no mechanism to update its parameters and will continue allocating capital based on relationships that may no longer hold. This limitation naturally leads to the use of approaches that can learn adaptive policies directly from data, specifically reinforcement learning, which frames portfolio management as a sequential decision-making problem, and evolutionary optimisation, which searches for robust solutions across multiple objectives simultaneously.

1.3 Personal Motivation & Project Framing

Having interned at Goldman Sachs and developed quantitative experience during my time at university in Canada, I was struck by discussions with portfolio managers at the Royal Bank of Canda, who described rebalancing allocations as infrequently as once a month. This prompted me to explore whether machine learning could introduce greater dynamism and adaptability to a process that in practice moves surprisingly slowly.

Investment management is a domain that has traditionally been slow to adopt adaptive, data-driven approaches, with many practitioners still relying on rules and models that assume stable market conditions. Recent advances in reinforcement learning and evolutionary computation offer a natural opportunity to address this, as both are capable of learning directly from experience rather than assuming a fixed model of the environment.

The project is scoped to a universe of 11 assets spanning equities, ETFs, which are baskets of assets that track an index or sector and are treated like a single stock, and defensive instruments, such as government bonds and cash, providing sufficient diversity to make portfolio allocation decisions meaningful while remaining computationally tractable.

1.4 The Formal Project Statement

This project implements and compares two adaptive approaches to portfolio management: a reinforcement learning agent based on Proximal Policy Optimisation (PPO) and an NSGA-II evolutionary optimiser. Both approaches use daily price data spanning up to 13.5 years from June 2012 to December 2025, divided into training, evaluation, and held-out test sets, with the test period covering significant market volatility and regime changes.

Performance is evaluated using cumulative returns, Sharpe ratio, which represents risk-adjusted returns, maximum drawdown, and transaction costs, with bootstrap significance testing used to assess whether results are statistically meaningful rather than attributable to chance. GARCH volatility forecasts are incorporated as an optional component of both the observation space and reward function, allowing a direct comparison between agents with and without access to forward-looking volatility information.

The best performing configuration in this project achieves a mean cumulative return of 1.2250 and a mean annualised Sharpe ratio of 1.5234 across 50 independent runs on a held-out test set of 207 trading days, outperforming both the NSGA-II variants and an equal-weight baseline.

The remainder of this report is structured with Section 2 reviewing relevant background literature, Section 3 detailing the methodology and system design, Section 4 describing implementation, and Section 5 presenting evaluation and conclusions.

2 Project Background and Context

2.1 Limitations of Markowitz Mean-Variance Optimisation

Markowitz mean-variance optimisation implicitly assumes that the joint distribution of asset returns is stationary, meaning the statistical relationships estimated from historical data will continue to hold over the investment horizon.

When this assumption is violated, as occurs during market regime changes, the estimated covariance matrix becomes misspecified, and the resulting portfolio weights may be poorly suited to the actual prevailing market conditions.

While extensions such as rolling-window estimation partially address this by periodically refitting the covariance matrix on recent data, this approach remains reactive rather than adaptive.

2.2 Reinforcement Learning for Portfolio Management

Reinforcement learning offers a natural framework for portfolio management by casting it as a Markov Decision Process, where an agent observes the current market state, selects portfolio weights as actions, and receives a reward signal based on the resulting portfolio performance.

The most influential work in this area is Jiang et al. (2017) [2], who proposed a deep reinforcement learning framework for portfolio management using an Ensemble of Identical Independent Evaluators architecture, tested on cryptocurrency markets with a 30-minute trading period. Their framework introduced the Portfolio Vector Memory, which preserves portfolio weights between trading steps, allowing the agent to account for transaction costs when making allocation decisions. Across three back-test experiments, their framework achieved at least four-fold returns over 50 days, significantly outperforming classical benchmarks.

However, these results must be interpreted carefully, as the evaluation is conducted entirely in cryptocurrency markets from late 2016 to early 2017, a period of sustained asset appreciation where the underlying growth of assets means almost any long position would have generated substantial returns [4]. A long position simply means owning an asset and profiting when its price rises. Furthermore, their reported Sharpe ratios are significantly deflated by their choice of Bitcoin as the benchmark rate, which is methodologically questionable given that Bitcoin is itself one of the most volatile assets in existence rather than a risk-free instrument. The evaluation period of 50 days is also notably short, limiting the extent to which results can be generalised to different market conditions.

This project uses PPO as the RL algorithm, which improves on earlier policy gradient methods through a clipped surrogate objective that prevents destabilising policy updates, making it particularly well-suited to the noisy reward signals characteristic of financial markets. It also extends the RL portfolio management paradigm to a daily equity setting, incorporating GARCH-based volatility forecasts as optional observation signals, evaluating performance over a longer and more realistic test period, and applying bootstrap significance testing to assess whether results are statistically meaningful rather than attributable to chance.

2.3 Multi-Objective Evolutionary Optimisation

Portfolio optimisation is inherently a multi-objective problem, as maximising return and minimising risk are competing objectives. This is because increasing expected return typically requires accepting greater risk, meaning no single solution simultaneously optimises both.

Rather than collapsing these objectives into a single weighted sum, multi-objective optimisation seeks a Pareto front. This is a set of solutions where no solution can be improved on one objective without worsening another, providing a complete picture of the available return-risk trade-offs.

NSGA-II, introduced by Deb et al. (2002) [1], is a fast elitist multi-objective genetic algorithm that has become one of the most widely used approaches for approximating the Pareto front in complex optimisation problems. The algorithm works by ranking solutions according to non-dominated sorting, where solutions that are not dominated by any other solution in the population form the first Pareto front, with subsequent fronts formed from the remaining solutions ranked similarly. To preserve diversity across the Pareto front, NSGA-II uses a crowding distance mechanism that favours solutions in less densely populated regions of the objective space, preventing the population from collapsing into a small cluster of similar solutions.

Kaucic et al. (2019) [3] demonstrate the effectiveness of NSGA-II for portfolio optimisation, applying it to multiple asset universes with competing risk measures including semi-variance and conditional value-at-risk, finding that NSGA-II consistently produces well-distributed Pareto fronts across all test problems. However, their evaluation is conducted entirely in-sample, meaning the optimisation and assessment are performed on the same data, which limits the extent to which their results can be generalised to unseen market conditions.

This project explicitly improves upon Kaucic et al. by evaluating all methods on a strictly held-out test set that is never used during training or model selection, providing a more reliable assessment of out-of-sample performance. Furthermore, the application of NSGA-II is extended to a rolling window setting, retraining the algorithm every 10 days on the most recent 40 days of market data, allowing the Pareto front to adapt to changing market conditions rather than remaining fixed throughout the test period.

Since NSGA-II produces a set of Pareto-optimal solutions rather than a single answer, a selection criterion is required to choose one solution for deployment. This project uses a distance-to-utopia approach, selecting the solution closest to the ideal point in normalised objective space as a compromise between return and maximum drawdown.

2.4 GARCH

GARCH, or Generalised Autoregressive Conditional Heteroskedasticity, is a class of time series models designed to capture the well-documented tendency of financial returns to exhibit volatility clustering, where periods of high volatility tend to be followed by further high volatility and calm periods tend to persist.

The GARCH(1,1) specification, which models the current conditional variance as a function of the previous period's squared return and the previous period's conditional variance, is the most widely used variant due to its simplicity and strong empirical performance across a wide range of asset classes and market conditions.

In this project, a GARCH(1,1) model is fitted independently per asset to produce daily one-step-ahead volatility forecasts, which are incorporated as an optional component of both the observation space and the LowVolatility reward function, providing the agent with a forward-looking signal about near-term uncertainty.

A limitation of fitting independent per-asset GARCH models is that they assume zero correlation between assets, meaning portfolio variance in the LowVolatility reward is computed from individual asset volatilities alone rather than the full covariance structure, which underestimates true portfolio risk when assets are positively correlated.

Dynamic Conditional Correlation GARCH, introduced by Engle (2002) [5], extends the univariate GARCH framework by modelling time-varying correlations between assets alongside their individual volatilities, producing a full dynamic covariance matrix that provides a more accurate portfolio risk signal than independent per-asset models. Whether the additional complexity of a full dynamic covariance estimate translates into improved portfolio performance in a reinforcement learning setting is an empirical question explored in this project.

3 Methodology and Specification

3.1 Problem Formulation

Both approaches addressed in this project treat portfolio management as a sequential decision-making problem, where at each time step an agent observes the current market state and selects a vector of portfolio weights to allocate capital across the available assets.

The asset universe consists of 11 assets: Apple, Microsoft, Alphabet (Google), Amazon, the S&P 500 ETF (SPY), the NASDAQ 100 ETF (QQQ), a long-term US Treasury Bond ETF (TLT), a Gold ETF (GLD), Coca-Cola, Boeing, and ExxonMobil, as well as a risk-free cash position, giving a total of 12 possible allocations.

The state observed by the agent at each time step consists of a 40-day window of asset prices, normalised by dividing by the first price in the observation window. This makes the observation scale-invariant, allowing the agent to learn from relative price movements rather than absolute price levels. This observation is optionally augmented with GARCH-based daily volatility forecasts for each asset, providing the agent with a forward-looking signal about near-term uncertainty.

The action space is a continuous vector of the same dimension as the number of assets, which is mapped to a valid portfolio weight vector through a softmax transformation, ensuring all weights are non-negative and sum to one, enforcing the long-only constraint without requiring explicit constraint handling.

For the RL agent, the reward signal is computed at each step as a function of portfolio performance, with four variants explored: pure return, log return, a concentration-encouraging variant using the Herfindahl-Hirschman Index, and a volatility-penalising variant using GARCH forecasts. Transaction costs are modelled as proportional to portfolio turnover at each rebalancing step, ensuring that excessive rebalancing is penalised in the portfolio value calculation.

3.2 Reinforcement Learning Agent Design

PPO is selected as the reinforcement learning algorithm due to its stability in continuous action spaces, achieved through a clipped surrogate objective that prevents excessively large policy updates which could destabilise training. This is a particularly important property given the noisy and non-stationary nature of financial return signals.

While algorithms such as DDPG and SAC are also designed for continuous action spaces, they are deterministic or require careful entropy tuning respectively, making them more sensitive to hyperparameter choices. PPO's stochastic policy and stable update rule make it more robust to the high variance reward signals characteristic of daily portfolio returns. An entropy bonus is included in the PPO objective to encourage exploration, preventing the policy from prematurely converging to a deterministic allocation strategy before sufficiently exploring the space of possible portfolio weights.

Rather than deploying the final trained policy, the best-performing model on the held-out evaluation set is selected via an evaluation callback, preventing overfitting to the training environment and ensuring the deployed policy generalises to unseen market conditions.

GARCH volatility forecasts are incorporated as an optional component of both the observation space and reward function, rather than a fixed feature, allowing a direct comparison between agents with and without access to forward-looking volatility information, isolating the contribution of the GARCH signal to overall performance.

It is important to note that once training is complete, the PPO policy is deployed in a fixed, offline manner, which means the model weights are not updated during the test period. Therefore, the agent responds dynamically to observations but does not continue learning from live market experience.

To assess the robustness of the learned policy across different market conditions, the best performing RL agent is evaluated on both a shorter and longer test period, investigating whether performance is consistent or dependent on the specific market regime covered by the test set.

3.3 Evolutionary Optimisation Design

NSGA-II is selected as the evolutionary optimisation algorithm due to its well-established effectiveness in approximating Pareto fronts for multi-objective problems, its computational efficiency through fast non-dominated sorting, its elitist selection mechanism which preserves high-quality solutions across generations, and its crowding distance mechanism which maintains diversity across the Pareto front. This prevents the population from converging to a narrow cluster of similar solutions and ensures a well-

distributed set of return-drawdown trade-offs is available for selection.

The two optimisation objectives are cumulative portfolio return and maximum drawdown, chosen because they capture the two primary concerns of a rational investor, which are generating growth and limiting downside loss, and because their inherent conflict produces a meaningful Pareto front across which different risk appetites can be represented.

A static variant of NSGA-II is implemented as a baseline, training on the evaluation set with a larger population of 80 and 50 generations to find a single fixed action that is then applied consistently throughout the entire test period, providing a direct comparison point for the rolling window variant. The rolling window variant retrains NSGA-II every 10 days on the most recent 40 days of market data, allowing the Pareto front and selected action to adapt as market conditions evolve. This directly addresses the stationarity assumption identified as a core limitation of classical approaches in Section 2.1.

The window size of 40 days captures approximately eight weeks of recent market behaviour, providing sufficient data for meaningful optimisation while avoiding the inclusion of stale data from market regimes that may no longer be relevant. Importantly, this matches the observation window size used by the RL agent, ensuring both approaches have access to the same historical context at each decision point and allowing for a fair comparison between methods.

The step size of 10 days balances adaptability against computational cost as retraining daily would be prohibitively expensive given the NSGA-II population size, while retraining less frequently would reduce the algorithm's ability to respond to changing market conditions.

A population size of 40 and 20 generations are used for the rolling variant, reduced from the static variant's 80 and 50 respectively, reflecting the computational constraints of retraining every 10 days across the full test period while still providing a sufficiently diverse population to approximate the Pareto front.

Since NSGA-II produces a set of Pareto-optimal solutions rather than a single action, a distance-to-utopia selection criterion is applied, choosing the solution with the smallest normalised distance to the ideal point where both return and maximum drawdown are simultaneously optimal. A weighted distance of 0.65 for return and 0.35 for drawdown is used, reflecting a preference for return-maximising solutions while still penalising excessive drawdown.

3.4 Post-Hoc Multi-Objective Selection

Comparing portfolio strategies using a single metric such as Sharpe ratio risks overlooking important differences in behaviour. A strategy that ranks highest on risk-adjusted return may simultaneously exhibit excessive concentration or larger drawdowns than alternative strategies that rank slightly lower. To address this, a post-hoc multi-objective dominance filter is applied across all candidate policies, including all RL variants and both NSGA-II variants.

This approach is particularly well-suited to the stochastic nature of PPO training, where different runs with the same reward function may produce meaningfully different policies due to random weight initialisation and stochastic gradient updates. Aggregating results across all variants before applying dominance filtering ensures the best-performing policies are identified regardless of training variance.

This multi-objective selection finds the subset of non-dominated policies where no policy is simultaneously better on all three objectives of return, mean HHI, and maximum drawdown. Return is included as the primary performance objective, maximum drawdown captures downside risk, and mean HHI measures portfolio concentration. Together these three objectives represent the key dimensions along which portfolio strategies can meaningfully differ, capturing growth, risk, and diversification simultaneously. An epsilon margin of 0.005 is applied when determining dominance, meaning a policy must be meaningfully better rather than marginally better on at least one objective to be considered non-dominating, preventing near-identical policies from being artificially separated by numerical noise.

The output of this step is a Pareto front of policies rather than a single winner, reflecting the broader multi-objective philosophy of the project and representing the set of strategies that offer genuinely distinct trade-offs between return, risk, and diversification. Each non-dominated policy is then reported and visualised individually, allowing the investor to assess which trade-off best suits their investment objectives.

3.5 Data and Evaluation Design

Daily adjusted closing prices are sourced from the Tiingo financial data API for 11 assets spanning the period from June 2012 to December 2025, providing approximately 13.5 years of market data that includes multiple distinct market regimes including a prolonged bull market, the COVID-19 crash and recovery of 2020, and the bear market conditions of 2022. Adjusted closing prices are used rather than raw closing prices to account for corporate actions such as stock splits and dividend payments, ensuring price continuity across the full data period and preventing artificial discontinuities in the return calculations.

For the longer test period, data is divided into four non-overlapping periods: a GARCH burn-in period of 251 days to initialise volatility model parameters, a training set of 2045 days for RL agent learning, an evaluation set of 275 days for both NSGA-II static training and RL model selection via the evaluation callback, and a held-out test set of approximately 747 days on which all final results are reported. The burn-in period is required because GARCH models need a sufficient history of returns to produce stable estimates.

Final performance is evaluated using three primary metrics: cumulative portfolio return, which measures total wealth

growth over the test period; annualised Sharpe ratio, which measures risk-adjusted return relative to daily T-bill rates sourced from the Federal Reserve; and maximum drawdown, which captures the worst peak-to-trough loss experienced during the test period.

To assess whether results are statistically meaningful rather than attributable to chance, a circular block bootstrap test is applied to the Sharpe ratio of each strategy, testing the null hypothesis that the true Sharpe ratio is less than or equal to zero. Sharpe ratio being zero is equivalent to the risk-free rate, which an investor can achieve without taking on any risk. A block size of 10 is used to preserve the autocorrelation structure of financial returns, which would be violated by standard i.i.d. bootstrap methods.

An equal-weight baseline is included as a benchmark, allocating capital uniformly across all 12 assets including cash throughout the test period, providing an uninformed reference point against which the performance of all adaptive approaches can be assessed.

In addition to the existing bootstrap significance test against a Sharpe ratio of zero, two complementary tests are applied to assess whether a strategy statistically outperforms the equal-weight baseline rather than simply generating positive returns: a paired circular block bootstrap test of the Sharpe ratio difference and the Memmel (2003) [7] corrected Jobson-Korkie test. Both tests are one-tailed, testing the null hypothesis that the strategy's Sharpe ratio is less than or equal to the baseline's, with a p-value below 0.05 providing evidence that the strategy's risk-adjusted return is meaningfully superior to the naive equal-weight allocation rather than merely above the risk-free rate.

4 Implementation

4.1 System Architecture Overview

Figure 1 in the Appendix shows the overall system architecture, which consists of four main stages: data preparation, volatility modelling, strategy training and testing, and post-hoc selection.

In the data preparation stage, daily adjusted closing prices are either fetched from the Tiingo API or loaded from CSV and passed through the GARCH volatility model, which produces per-asset daily volatility forecasts that are precomputed and cached before any training begins. This ensures volatility estimates, which are computationally expensive, are available consistently across all splits.

Both the RL and NSGA-II pipelines interact with a shared PortfolioEnv environment, which is a custom environment class I built on the Gymnasium interface that handles observation window construction, action execution, transaction cost calculation, and performance tracking in a unified framework. Gymnasium is the standard Python interface for reinforcement learning environments, defining the observation space, action space, reset, and step functions that any compliant RL library can interact with.

The RL pipeline trains a PPO agent using Stable-Baselines3, selecting the best model via an evaluation callback before deploying on the test set. Stable-Baselines3 is a well-established open-source library providing reliable, research-grade implementations of PPO and other deep RL algorithms with a consistent training API. The NSGA-II pipeline uses the pymoo library to optimise portfolio actions directly, implemented in both static and rolling window variants.

Following evaluation on the held-out test set, performance metrics are computed for all strategies and passed through the post-hoc multi-objective dominance filter, which identifies the Pareto front of non-dominated policies that are then individually visualised or reported to the evaluation framework.

4.2 Portfolio Environment

PortfolioEnv provides a unified interface through which both the PPO agent and NSGA-II algorithm interact with market data, ensuring both methods are evaluated under identical conditions. At each daily time step, the environment executes the following sequence: the agent's raw action vector is mapped to portfolio weights via softmax, transaction costs are calculated as a proportion of the total turnover relative to the previous day's weights, asset returns are computed from consecutive daily price observations, and the portfolio value is updated accordingly.

The reward signal for PPO is computed via a RewardFunction interface that is passed into the environment at instantiation through dependency injection. This allows different reward variants to be tested without modifying the environment.

For NSGA-II, the reward function is not used. Transaction costs are calculated as half the sum of absolute weight changes, which is the standard turnover formula, multiplied by the portfolio value and a transaction fee of 0.1%. This reflects the one-way cost of rebalancing rather than double-counting buys and sells, and is consistent with realistic brokerage costs for institutional equity trading.

The reset function initialises the portfolio to a fully cash position with a portfolio value of 1.0 and accepts optional parameters for the initial portfolio value and previous prices, allowing the environment to be initialised mid-episode with a carried-over portfolio state, which is a feature required by the rolling NSGA-II implementation.

Throughout each episode the environment tracks portfolio value, portfolio weights, daily returns, and transaction costs at each step, which are accessible via the get_metric_summaries method that returns a standardised dictionary of MetricSummary objects containing both the full time series and aggregate values for each metric.

4.3 GARCH Volatility Model

Volatility forecasting is implemented through two custom classes I wrote, MultiAssetGARCH and GARCHVolatilityModel, where MultiAssetGARCH maintains

one independent GARCHVolatilityModel instance per asset. This produces daily volatility forecasts that are precomputed across the full dataset before any training begins and cached as numpy arrays for efficient loading during experiments.

Each per-asset model is specified as a GARCH(1,1) with a normal error distribution, implemented using the arch library, with returns scaled by a factor of 100 before fitting to improve numerical stability during parameter estimation. Each model maintains a rolling buffer of the most recent 250 returns using a deque data structure, ensuring the GARCH parameters are estimated on a fixed-length recent history rather than the full return series. The full return series with an ever-expanding window would cause parameters to average across all historical regimes rather than reflecting current conditions.

To manage computational cost, the GARCH model is refitted every 10 steps rather than at every time step, with the most recently fitted model used to produce forecasts in between refits. This is a trade-off between parameter freshness and computational tractability given that GARCH parameters typically evolve slowly relative to daily returns.

A burn-in period of 251 trading days is enforced before any GARCH forecasts are used in training, ensuring the rolling buffer is fully populated and the model has converged to stable parameter estimates before volatility signals are introduced into the observation space. At each step, the model produces a one-step-ahead volatility forecast by extracting the square root of the forecasted variance from the fitted model, scaled back from percentage returns to decimal form by dividing by 100, producing an 11-dimensional vector of daily volatility estimates.

As noted in Section 2.4, a limitation of fitting independent per-asset GARCH(1,1) models is that they assume zero correlation between assets, underestimating true portfolio risk when assets are positively correlated. To address this, a DCC-GARCH implementation was developed using the arch library, fitting a dynamic conditional correlation model across all 11 assets simultaneously to produce a full time-varying covariance matrix at each step. The resulting covariance matrix was used to compute a more accurate portfolio variance estimate in the LowVolatility reward function, replacing the diagonal approximation with the full quadratic form using the portfolio weight vector and the dynamic covariance matrix.

However, evaluation on a subset of runs showed that DCC-GARCH did not produce a measurable improvement in portfolio performance over the independent GARCH(1,1) baseline, likely because the additional complexity of the full covariance signal increased the noise in the reward without providing a sufficiently cleaner learning gradient for the PPO agent to exploit within the available training budget. As a result, the independent GARCH(1,1) specification was retained for all final experimental configurations, with DCC-GARCH noted as a direction for future work in Section 6.

4.4 PPO Training Pipeline

The PPO agent is implemented using the Stable-Baselines3 library with an MLP policy network, trained on the training environment for 20,000 timesteps with an entropy coefficient of 0.05 to encourage exploration during the early stages of training.

An EvalCallback is configured to evaluate the current policy on the held-out evaluation environment every 2,000 timesteps, saving the best-performing model checkpoint locally. This ensures the deployed policy is selected based on evaluation performance rather than final training performance, which may have degraded due to overfitting and not generalise well. Following training, the best saved model is loaded and deployed on the test environment in deterministic mode, meaning the policy outputs the action with the highest probability rather than sampling from the action distribution, ensuring consistent and reproducible test results.

The training environment is initialised with the training price data and corresponding precomputed GARCH volatility arrays, with the observation window aligned to begin after the GARCH burn-in period to ensure all observations contain valid volatility forecasts where required. Eight experimental variants are trained in total, with the four reward functions each running twice, once with GARCH volatility included in the observation space and once without, producing a systematic 2x4 experimental design that isolates the contribution of the volatility signal independently of the reward function choice.

4.5 NSGA-II Implementation

The NSGA-II optimiser is implemented using the pymoo library with a custom ElementwiseProblem subclass, which is pymoo's base class for problems where each candidate solution is evaluated independently rather than in batches, with PortfolioNSGA2RollingProblem for the rolling variant and PortfolioNSGA2StaticProblem for the static variant.

Both subclasses wrap the PortfolioEnv environment to evaluate candidate solutions, where each solution is represented as a continuous vector of length 12 with one value per asset including cash. The values, bounded by −10 to 10, are passed through the same softmax transformation used by the RL agent to produce valid portfolio weights, ensuring the action space is identical across both methods.

For each candidate solution, the problem class resets the environment and simulates the full episode using a fixed action, applying the same portfolio weights at every step, then extracts the final portfolio value and maximum drawdown as the two objectives, negating both for minimisation as required by pymoo's convention.

The rolling variant handles the transition between windows by passing the final portfolio weights and portfolio value from the previous test window into the next training environment via the reset function's initial_weights and initial_portfolio_value parameters, ensuring portfolio continuity across retraining boundaries. The rolling variant iterates through the test period in steps of 10 days, training on the preceding 40-day window and evaluating on the next 10-day window, with the best action selected via the

distance-to-utopia criterion applied to the normalised Pareto front objectives at each iteration.

The static variant trains on the evaluation set rather than the training set, using a larger population of 80 and 50 generations to find a single fixed action vector that is then applied consistently across the entire test period without any retraining.

4.6 Reward Functions

All reward functions inherit from a common RewardFunction base class that defines a standard interface, accepting portfolio value before and after the step, current weights and GARCH volatility forecasts as keyword arguments. This allows new reward functions to be added without modifying the environment.

PureReturn computes the single-step percentage return of the portfolio, providing a direct and unmodified learning signal that incentivises the agent to maximise wealth growth without any explicit penalty for risk or concentration. LogReturn computes the natural logarithm of the portfolio value ratio, which penalises large losses more heavily than equivalent gains due to the asymmetric nature of logarithmic scaling, providing a more conservative learning signal than pure return.

HighHHI augments the pure return signal with a positive HHI term weighted by alpha of 0.15, explicitly incentivising portfolio concentration by rewarding the agent for allocating heavily to fewer assets. The counterintuitive result that this produces the highest Sharpe ratio in testing is discussed in Section 5.

LowVolatility augments the pure return signal with a negative portfolio variance term weighted by alpha of 0.3, computing portfolio variance as the weighted sum of squared GARCH volatility forecasts. This penalises allocations to assets with high predicted volatility and incentivises the agent to reduce exposure before volatile periods. This variance calculation assumes zero correlation between assets, meaning portfolio variance is computed from individual asset volatilities alone rather than the full covariance matrix, which is a simplification that reduces computational complexity but underestimates true portfolio risk when assets are positively correlated.

The NSGA2 reward class raises NotImplementedError when called, existing purely to satisfy the RewardFunction interface required by PortfolioEnv since NSGA-II does not use a reward signal.

4.7 Evaluation Framework

Performance tracking is centralised through a MetricSummary dataclass that stores the full time series, aggregate value, direction of optimisation, and reward function name for each metric, providing a consistent interface for both per-step tracking during episodes and aggregate reporting after testing is complete.

The annualised Sharpe ratio is calculated using daily Treasury bill rates sourced from the Federal Reserve, forward-filled from monthly observations to daily frequency using business day resampling, with excess returns computed as daily portfolio returns minus the corresponding daily risk-free rate before annualising by multiplying by $\sqrt{252}$.

The Herfindahl-Hirschman Index is computed at each step as the sum of squared portfolio weights, producing a value between 1/n for a perfectly equal allocation and 1.0 for full concentration in a single asset. This is tracked as a time series and reported as a mean across the test period.

Maximum drawdown is tracked as a running time series by maintaining a peak portfolio value at each step and computing the percentage decline from that peak, with the aggregate reported as the largest decline across the series, representing the worst loss experienced relative to the portfolio's highest point up to that date.

Bootstrap significance testing, which is shown in Figure 2 in the Appendix, is implemented using the CircularBlockBootstrap class from the arch library, applying 1,000 bootstrap iterations with a block size of 10 to the portfolio return series, computing the Sharpe ratio for each resampled series and reporting a 95% confidence interval and a one-tailed p-value testing the null hypothesis.

The equal-weight baseline, which serves as a strong benchmark, is implemented by initialising a PortfolioEnv instance on the test set and stepping through it with a fixed uniform weight vector of 1/12 across all 12 assets including cash, using the same environment and metric calculation pipeline as the adaptive strategies to ensure a fair comparison.

Two additional tests are applied to assess whether a strategy statistically outperforms the equal-weight baseline. The paired bootstrap test resamples both the strategy and baseline return series jointly using the same block indices, preserving their pairing across bootstrap iterations and ensuring the variance estimate of the difference accounts for the correlation between the two series, which share the same market environment. A one-tailed p-value is computed as the proportion of bootstrap samples in which the resampled Sharpe difference is less than or equal to zero, testing the null hypothesis that the strategy's Sharpe ratio is no greater than the baseline's.

The Jobson-Korkie test is implemented using the Memmel (2003) [7] correction to the original Jobson and Korkie (1981) formulation, which addresses a bias in the original variance estimator. The test derives an asymptotic z-statistic for the difference between the two daily Sharpe ratios, standardised by the square root of the Memmel variance estimate, which accounts for the correlation between the two return series through the sample correlation coefficient. A one-tailed p-value is then obtained from the standard normal distribution. Unlike the bootstrap test, the Jobson-Korkie test assumes normally distributed and serially uncorrelated returns, a condition that daily equity returns are known to violate to some degree. The two tests are thus used together, with agreement between them providing stronger evidence than either test individually.

5 Results

5.1 Overview

Results are reported across four experimental configurations: Large, Augmented Large, Small, and Augmented Small. The Large configuration trains on daily adjusted closing price data spanning June 2012 to December 2025, using a 65/10/25 train–eval–test split, providing over 13 years of market history across multiple distinct regimes. The Small configuration uses a shorter window from June 2017 to December 2024 with a 70/15/15 split. The Augmented variants of each configuration extend the observation space to include the agent's current portfolio weights alongside the normalised price window, allowing the policy to condition its allocation decisions on its existing position.

Each configuration was run 50 times with independent random seeds to account for the stochastic nature of PPO training, with results reported as mean ± standard deviation across all 50 runs and the single best run by Sharpe ratio also noted.

The equal-weight baseline, described in Section 4.7, serves as the primary benchmark against which the configurations are compared. Full results across all four configurations and the baseline are reported in Tables 1–3 in the Appendix.

5.2 Large Configuration

The Large configuration, tested on 747 days of returns, produces the strongest absolute returns of all four experimental settings, with a mean cumulative return of 1.9541 ± 0.2536 across 50 runs and a best run by Sharpe ratio achieving a cumulative return of 2.2132 and an annualised Sharpe ratio of 1.6845 (Table 1). The highest observed return is 2.7240, however the annualised Sharpe ratio of 1.5982 is lower, meaning it is not the best performer when adjusted for risk.

The mean annualised Sharpe ratio of 1.2650 ± 0.1423 and best run Sharpe of 1.6845 indicate that these returns are achieved with meaningful risk adjustment, not simply by taking on excessive market exposure (Table 1). The mean maximum drawdown of −0.1499 ± 0.0341 reflects moderate downside risk across runs, with the best Sharpe run limiting drawdown to −0.0881, suggesting that well-trained policies in this configuration are capable of meaningfully controlling losses during adverse market periods (Table 1).

Statistical significance is robust across the full set of runs, with all 50 achieving a bootstrap p-value below 0.05 and a mean p-value of 0.0117 ± 0.0088, providing strong evidence that strategies in this configuration generate returns meaningfully above the risk-free rate rather than by chance (Table 3).

Across the 50 runs, PureReturn is selected as the dominant method most frequently by the post-hoc Pareto filter, winning 19 runs, followed by LogReturn with 12, LowVolatility with 11, and HighHHI with 8, suggesting that simpler, unmodified return signals produce the most consistently robust policies when training data is abundant. Notably, NSGA-II is not selected as the dominant method in any of the 50 runs, indicating that when the RL agent has access to over 13 years of training data it consistently produces policies that dominate the evolutionary approach across all three objectives of return, drawdown, and concentration.

A concrete example of this adaptive behaviour can be observed between day 550 and day 600 of the test period, which corresponds to the sharp global equity sell-off of August 2024 triggered by the unexpected Bank of Japan rate rise. Figure 3 in the Appendix shows the LogReturn with GARCH agent moving away from equity-heavy positions and increasing exposure to defensive assets, namely cash and gold, in the surrounding period, illustrating the practical value of a dynamically rebalancing policy over a static allocation in response to sudden market stress.

The mean transaction cost across all 50 runs of 9.378% ± 7.906% reflects considerable variance in policy aggressiveness, with the high standard deviation driven primarily by runs in which HighHHI or LowVolatility variants are selected by the dominance filter, as both reward functions tend to incentivise more frequent rebalancing relative to PureReturn and LogReturn (Table 2).

Against the equal-weight baseline, the Large configuration produces a mean bootstrap Sharpe difference of +0.049 but neither the paired bootstrap nor Jobson-Korkie test finds this significant, with mean p-values of 0.535 and 0.523 respectively, indicating that while strategies reliably beat the risk-free rate, consistent outperformance over the baseline is harder to establish statistically.

5.3 Augmented Large

The Augmented Large configuration, which extends the observation space to include current portfolio weights, produces a mean cumulative return of 1.8814 ± 0.2465 and a best run by Sharpe ratio achieving a return of 2.3007 and an annualised Sharpe of 1.5096 (Table 1). Compared to the non-augmented Large configuration, mean return falls slightly from 1.9541 to 1.8814 and mean Sharpe ratio from 1.2650 to 1.2316, suggesting that the addition of portfolio weights to the observation space does not improve performance and may introduce complexity that the policy network struggles to exploit (Table 1).

Statistical significance remains strong, with 49 out of 50 runs achieving a bootstrap p-value below 0.05 and a mean p-value of 0.0117 ± 0.0085, virtually identical to the non-augmented Large configuration and confirming that the strategies are consistently generating returns above the risk-free rate (Table 3).

The method selection distribution is considerably more dispersed than in the Large configuration, with no single reward function dominating. HighHHI wins only 2 runs compared to 8 in the non-augmented setting, potentially indicating that when the agent can already observe its current allocations, the explicit concentration incentive becomes less necessary, as the policy can learn to maintain focused positions through direct observation of its weights rather than through reward shaping. NSGA-II is

again not selected as the dominant method in any of the 50 runs, consistent with the Large configuration finding that a well-trained RL agent with sufficient data reliably dominates the evolutionary approach.

The mean HHI of 0.1984 ± 0.1746 is notably lower than the Large configuration's 0.2697, suggesting that augmented policies tend toward more diversified allocations on average, which is consistent with an agent that can observe its current weights and actively avoid over-concentration (Table 2). Mean transaction costs of 7.047% ± 9.160% are lower than the Large configuration with a mean of 9.378%. This is consistent with the intended effect of augmentation, suggesting that observing current portfolio weights encourages more incremental rebalancing on average (Table 2).

5.4 Small

The Small configuration, tested on 207 days of returns, produces a mean cumulative return of 1.2250 ± 0.0557 and a best run by Sharpe ratio achieving a return of 1.3769 and an annualised Sharpe of 2.5766 (Table 1). The mean Sharpe ratio of 1.5234 ± 0.3160 is higher than the Large configuration's 1.2650, indicating that despite the shorter test period the strategies achieve strong risk-adjusted returns, with the best run reaching a Sharpe of 2.5766 and a maximum drawdown of only −0.0597.

Statistical significance is considerably harder to establish in this configuration due to the shorter test period providing a smaller sample of daily returns for the bootstrap procedure. With fewer trading days, the confidence intervals around each Sharpe estimate are wider, making it more difficult to reject the null hypothesis that the true Sharpe ratio is zero, and as a result only 14 out of 50 runs achieve a bootstrap p-value below 0.05 with a mean p-value of 0.0803 ± 0.0399 (Table 3). Rolling NSGA-II appears as the dominant method for the first time across any configuration, winning 7 out of 50 runs, with these runs sharing a consistent profile of high concentration averaging an HHI of 0.59, very low transaction costs averaging 1.1%, and moderate Sharpe ratios averaging 1.22, none of which achieve statistical significance at the 5% level. Crucially, Rolling NSGA-II dominates only in runs where the RL agent has underperformed rather than in runs where NSGA-II has excelled, with the mean Sharpe across non-NSGA-II runs of 1.5723 considerably higher than the 1.2234 observed in NSGA-II-dominated runs, suggesting that NSGA-II fills a gap only when RL training is unsuccessful.

Mean transaction costs of 2.734% ± 1.230% are substantially lower than both Large configurations, reflecting the shorter test period and the prevalence of lower-turnover methods including NSGA-II being selected by the dominance filter (Table 2).

Against the equal-weight baseline, the Small configuration produces a stronger mean Sharpe difference of +0.410, though this remains insignificant on average with mean p-values of 0.318 and 0.356. The best run achieves a bootstrap baseline p-value of 0.026, with the Jobson-Korkie test returning 0.087 for the same run.

5.5 Augmented Small

The Augmented Small configuration produces a mean cumulative return of 1.2244 ± 0.0724 and a best run by Sharpe ratio achieving a return of 1.3001 and an annualised Sharpe of 2.5564, with the highest observed return across all 50 runs being 1.5590 (Table 1). The mean Sharpe ratio of 1.4117 ± 0.3333 is lower than the non-augmented Small configuration's 1.5234, suggesting that as with the Large configurations, the addition of portfolio weights to the observation space does not improve risk-adjusted performance and may hinder policy convergence.

Statistical significance is the weakest of all four configurations, with only 8 out of 50 runs achieving $p < 0.05$ and a mean p-value of 0.0940 ± 0.0423, reflecting the compounding difficulty of a shorter test period and a larger observation space (Table 3). Rolling NSGA-II is the dominant method in 13 out of 50 runs, its strongest showing across all configurations, again exclusively in runs where the RL agent has underperformed, with the mean Sharpe across non-NSGA-II runs of 1.4612 considerably higher than the 1.2709 observed in NSGA-II-dominated runs. All 13 NSGA-II-dominated runs fail to achieve statistical significance at the 5% level, with p-values uniformly above 0.09, further confirming that the rolling evolutionary approach is not a reliable standalone strategy but rather a fallback when RL training fails to converge.

Mean transaction costs of 2.368% ± 1.662% are the lowest of all four configurations, consistent with both the high prevalence of Rolling NSGA-II selections and the intended effect of portfolio weight augmentation encouraging more incremental rebalancing, making it difficult to isolate the contribution of either factor (Table 2).

5.6 Cross-Configuration Comparison

Taken together, the results across all four configurations reveal several consistent patterns that speak directly to the central research questions of this project. The Small configurations produce higher mean Sharpe ratios than their Large counterparts, however this likely reflects the character of the Small test period rather than genuinely superior strategy quality. Small's shorter test period is relatively bullish and captures fewer adverse market regimes, meaning strategies are evaluated under more favourable conditions than the full market history covered by the Large test period.

Observation augmentation with portfolio weights does not improve risk-adjusted performance in either data regime, with mean Sharpe ratio falling from 1.2650 to 1.2316 in the Large configurations and from 1.5234 to 1.4117 in the Small configurations, suggesting that the additional observation dimensionality requires more training data to learn useful representations from than the fixed training budget allows. However, augmentation does consistently reduce mean transaction costs in both regimes, from 9.378% to 7.047% in the Large configurations and from 2.734% to 2.368% in the Small configurations, which is consistent with its intended design goal of encouraging more incremental rebalancing.

Rolling NSGA-II only competes with the RL agent in the Small configurations, and even there exclusively in runs where the RL agent has underperformed, never in runs where NSGA-II has itself excelled, confirming that the evolutionary approach cannot match a well-trained adaptive policy but provides a meaningful safety net when RL training is unsuccessful. In the Large configurations, where training data is sufficient for PPO to converge reliably, the RL agent dominates in every single run, which directly validates the core motivation of this project: that reinforcement learning's ability to learn adaptive behaviour from experience provides a fundamental advantage over static optimisation approaches in non-stationary markets.

Finally, transaction costs represent a meaningful performance consideration across all configurations, with high-turnover runs in the Large configurations reaching over 20% of initial capital. However, as these are calculated using a realistic institutional brokerage fee of 0.1% per rebalancing step, they represent genuine net returns rather than an optimistic upper bound, meaning the reported returns already fully account for the cost of dynamic rebalancing.

The baseline comparison results are consistent across Large and Small with positive mean Sharpe differences, but no statistical significance on average under either test. This is partly attributable to the strength of the baseline itself as the asset universe was selected retrospectively from instruments known to have performed strongly over the data period, inflating the equal-weight return and raising the bar for demonstrating statistically significant outperformance.

6 Concluding Remarks

This project implements and evaluates two adaptive approaches to portfolio management, a PPO-based reinforcement learning agent and an NSGA-II evolutionary optimiser, demonstrating that reinforcement learning can produce statistically robust, risk-adjusted portfolio strategies that consistently outperform a static evolutionary approach when provided with sufficient training data.

The Large configuration results are the strongest evidence for this conclusion: across 50 independent runs, the post-hoc Pareto dominance filter selects an RL policy in every single case, all 50 achieve statistical significance at the 5% level, and the best run achieves an annualised Sharpe ratio of 1.6845 and a cumulative return of 2.2132, with all returns reported net of a realistic institutional brokerage fee of 0.1%.

The post-hoc multi-objective dominance framework, which selects non-dominated policies across return, maximum drawdown, and portfolio concentration simultaneously, proves particularly valuable in a stochastic training setting where different PPO runs with the same reward function can produce meaningfully different policies, ensuring the best available trade-off is identified regardless of training variance. Rolling NSGA-II provides a meaningful secondary contribution as a safety net. In configurations where RL training is less reliable due to limited data, the evolutionary approach fills the gap in a consistent and interpretable way, producing low-turnover, concentrated allocations that remain competitive when the RL agent fails to converge. However, several limitations of the current implementation warrant acknowledgement.

Most significantly, the PPO agent is deployed in a fixed, offline manner, meaning its weights are frozen after training and the policy does not continue learning from live market experience during the test period. This means the agent responds dynamically to new observations, but cannot adapt its underlying parameters to structural market changes that occur after training ends. The LowVolatility reward function computes portfolio variance as the weighted sum of individual asset GARCH forecasts, which assumes zero correlation between assets and consequently underestimates true portfolio risk when assets are positively correlated, as is typically the case during market stress when correlations tend to spike.

The observation augmentation experiment suggests that including portfolio weights in the observation space does not improve risk-adjusted performance within the current training budget, though it does reduce transaction costs, indicating that the design choice may be sound in principle but requires a longer training horizon or larger network capacity to exploit effectively.

An impactful extension would be to incorporate periodic retraining of the PPO agent on a rolling window of recent market data, analogous to the approach already used by Rolling NSGA-II, which would allow the policy weights to adapt to structural market changes without abandoning the PPO framework and directly address the core limitation of offline deployment in non-stationary markets. It is worth noting that this extension would have the most impact over longer test periods, where a frozen policy trained on historical data cannot reasonably be expected to remain reliable as market conditions evolve over many years or even decades. Finally, the diagonal GARCH variance approximation in the observation window and LowVolatility reward could be replaced with an alternative dynamic covariance specification. DCC-GARCH was tested in this project but did not produce a measurable improvement within the available training budget, suggesting that more sophisticated covariance modelling alone is insufficient and that architectural changes to how the risk signal is consumed by the policy network may be required.

From a practical standpoint, the results of this project suggest that reinforcement learning is a viable and promising approach to adaptive portfolio management. All configurations are evaluated on a realistic asset universe, use a genuine institutional brokerage fee of 0.1%, and are tested on a strictly held-out period, with the strongest configurations producing Sharpe ratios and returns that compare favourably with passive benchmarks, providing a credible foundation for further development toward live deployment.

References

[1] Deb, K., Pratap, A., Agarwal, S. and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), pp.182–197.

[2] Jiang, Z., Xu, D. and Liang, J. (2017). *A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem*. [online] arXiv.org. Available at: https://arxiv.org/abs/1706.10059.

[3] Kaucic, M., Moradi, M. and Mirzazadeh, M. (2019). Portfolio optimization by improved NSGA-II and SPEA 2 based on different risk measures. *Financial Innovation*, 5(1). doi:https://doi.org/10.1186/s40854-019-0140-6.

[4] Cross, J.L., Hou, C. and Trinh, K. (2021). Returns, volatility and the cryptocurrency bubble of 2017–18. *Economic Modelling*, 104, p.105643. doi:https://doi.org/10.1016/j.econmod.2021.105643.

[5] Engle, R. (2002). Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models. *Journal of Business & Economic Statistics*, [online] 20(3), pp.339–350. Available at: https://www.jstor.org/stable/1392121?seq=1.

[6] Jie, L.H. (2025). *Bank of Japan faces a policy dilemma as government bond yields keep hitting new highs*. [online] CNBC. Available at: https://www.cnbc.com/2025/12/04/japan-record-high-jgb-yields-boj-policy-rate.html.

[7] Memmel, C. (2003). Performance Hypothesis Testing with the Sharpe Ratio. *Finance Letters*, 1(1), pp.21–23.

Appendix

Table 1 – Core Performance Characteristics (Return, Sharpe Ratio, Maximum Drawdown)

	Return			Sharpe Ratio			Maximum Drawdown		
	Mean	Std	Best Run	Mean	Std	Best Run	Mean	Std	Best Run
Baseline Large	1.73923			1.18220			-0.14093		
Baseline Small	1.13589			1.07720			-0.06703		
Large	1.95408	0.25355	2.21318	1.26498	0.14231	1.68452	-0.14994	0.03408	-0.08806
Small	1.22504	0.05572	1.37689	1.52342	0.31596	2.57660	-0.07684	0.02237	-0.05974
Augmented Large	1.88137	0.24648	2.30071	1.23156	0.10962	1.50959	-0.14846	0.03993	-0.14309
Augmented Small	1.22442	0.07244	1.30010	1.41172	0.33329	2.55636	-0.08378	0.02334	-0.04321

Table 2 – Portfolio Characteristics (Mean HHI, Transaction Cost Percentage)

	Mean Herfindahl-Hirschman Index			Transaction Cost Percentage of Initial Capital		
	Mean	Std	Best Run	Mean	Std	Best Run
Baseline Large	0.083333			0.0%		
Baseline Small	0.083333			0.0%		
Large	0.269658	0.196856	0.300634	9.378368%	7.90574	15.222657%
Small	0.404917	0.172594	0.531487	2.733755%	1.230314	4.049399%
Augmented Large	0.198429	0.174609	0.396147	7.04744%	9.159909	22.126537%
Augmented Small	0.387385	0.191354	0.53593	2.367994%	1.661786	3.119769%

Table 3 – Statistical Significance (Bootstrap p-value, Sharpe Ratio Difference Against Baseline, Jobson-Korkie p-value)

	Bootstrap p-value, H_0: Sharpe Ratio ≤ 0			Baseline Sharpe Ratio Difference	Bootstrap p-value, H_0: Sharpe Ratio ≥ Baseline			Jobson-Korkie p-value, H_0: Sharpe Ratio ≥ Baseline		
	Mean	Std	Best Run	Mean	Mean	Std	Best Run	Mean	Std	Best Run
Baseline Large	0.014									
Baseline Small	0.147									
Large	0.01174	0.00885	0.0	0.049008	0.53466	0.20186	0.154	0.52251	0.15971	0.24913
Small	0.08032	0.03993	0.011	0.410424	0.31824	0.15206	0.026	0.35599	0.12127	0.08691
Augmented Large	0.01168	0.00847	0.001							
Augmented Small	0.094	0.04233	0.001							

Figure 1 – System Architecture Diagram

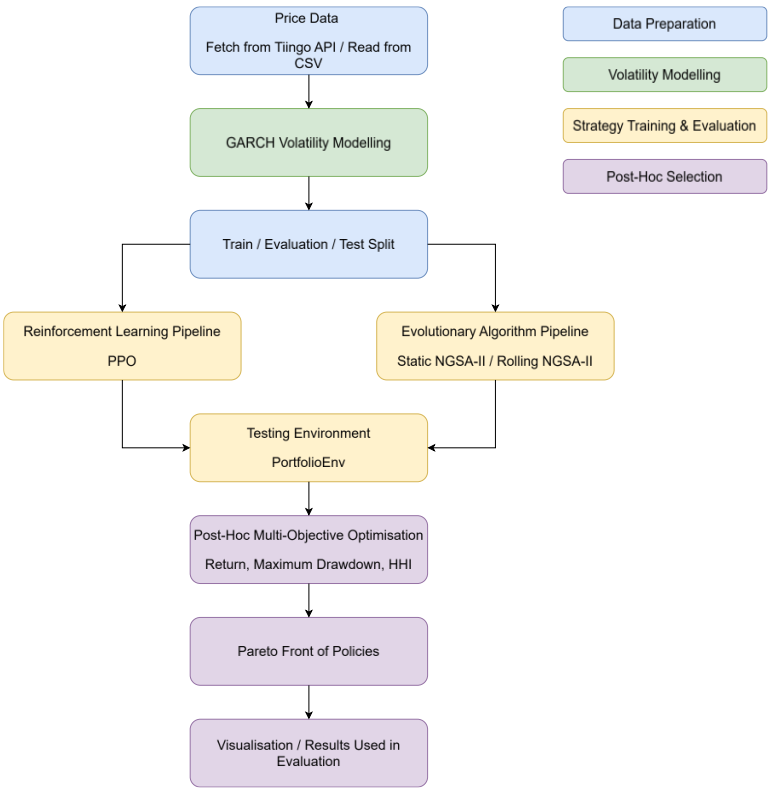


Figure 2 – Circular Block Bootstrap Diagram

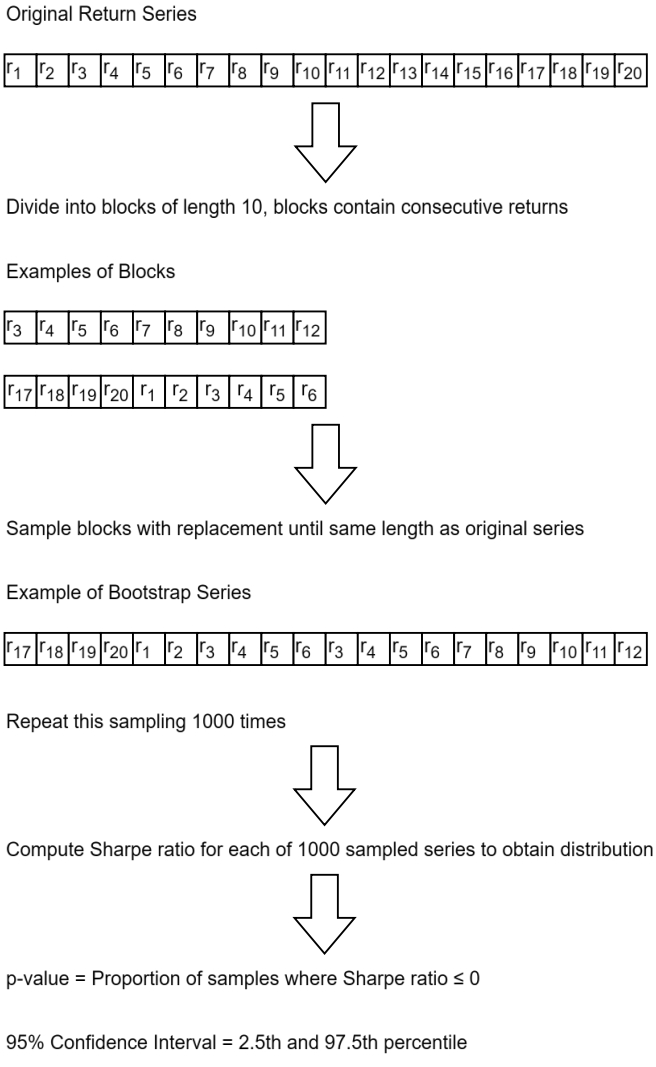


Figure 3 – LogReturn with Garch Responding Defensively to August 2024 Global Sell-Off

